`rewrite`: a programming language that learns its mathematics from rule files

Rainer Heintzmann
Friedrich-Schiller-Universität Jena, Germany
rainer.heintzmann@uni-jena.de

Abstract

`rewrite` is a small interpreted language for symbolic and numerical mathematics, written in the early 1990s and revived in 2026. Unlike most computer algebra systems, its kernel contains no mathematics: data types, operators, the laws of algebra, differentiation and integration rules are all defined in plain-text include files and applied by term rewriting. Rule sets ("algebras") are first-class objects that can be combined into sequences and fixpoint iterations. A `LIFT` statement transfers all rules of one set of operations to another, so that, for example, the rules of an abelian group written for $+$ also apply to $\cdot$ in a field. Numerical evaluation is interleaved with rewriting: as soon as all arguments of a function are numbers, an attached C function computes the value. We describe the design of the language, its revival on current 64-bit compilers, and its extension by static subsorts, which add exact integers and fractions as sorts below the real numbers without changing any rule of the library, and sorts of non-negative and positive numbers for which rules like $\sqrt{x^2} = x$ hold. As a case study, the rule-based integrator Rubi was translated into `rewrite`. A translator converts 96% of Rubi's rules automatically; for this, the language gained ordered rule lists with a backtracking matcher for optional pattern parts, commutative operators and condition lists, symbolic helper functions and special functions. On 1540 integration problems from four independent textbook collections, 90.4% of the results are verified correct by numerical differentiation.

1 Introduction

Computer algebra systems such as Mathematica [17] or SymPy [10] come with a large body of built-in mathematical knowledge. A user can add rules, but the core simplification and the calculus routines are part of the system and are mostly hidden from the user. Term rewriting systems such as Maude [5] take the opposite view: they provide a general rewriting engine and leave the mathematics to the user, but they are usually not aimed at numerical work.

`rewrite` sits between these positions. Its interpreter knows how to parse expressions with user-defined operators, how to match and replace terms, and how to call C functions for numerical values. Everything else is defined in include files: the sort `real`, the operators `+`, `*` and `^`, the group, ring and field laws, the differentiation rules, and numerical algorithms such as Simpson integration. The language was designed by the author between 1992 and 1996 on Unix and the Amiga, on top of a scanner and parser framework by Axel T. Schreiner. In 2026 it was ported to current compilers and extended with a test suite, exact numbers and a rule-based integration library.

The main contributions of this paper are:

- a description of a language in which mathematics is a library of rewrite rules, including the `LIFT` mechanism for transferring rules between algebraic structures;
- the interleaving of symbolic rewriting and numerical evaluation, which allows numerical algorithms to be written as rewrite rules;

- static subsorts with value conversion, which add exact integers and fractions to the language as a library, with the rules of the real numbers inherited through `LIFT`;
- an automatic translation of the rule-based integrator Rubi [11] into `rewrite`, with a measurement on independent test problems and a discussion of what the translation requires from the host system.

2 Language design

2.1 Sorts, operators and numbers

A program first declares sorts (types) and operators. The following complete program defines real numbers with three operators and evaluates expressions:

```
SORT real                                   # a data type
NUM real                                    # numbers typed in have sort real
FUNC + : real,real >> real LA2 ATT plus     # left-associative, priority 2
FUNC * : real,real >> real LA3 ATT mal      # binds tighter than +
FUNC ^ : real,real >> real RA  ATT hoch     # right-associative
1+2*3                                       # prints 7
2^3^2                                       # prints 512
CONST x,y : real
x+y*2                                       # prints x + y * 2
```

The format words `LA1`–`LA5`, `RA` and `RU` define left-associative binary operators of five priorities, right-associative operators and right unary operators such as the factorial. `ATT name` attaches a C function from a table of user functions. It is called whenever all arguments of the function are numbers; otherwise the term stays symbolic. `OTHER` adds further meanings to an existing name for other argument sorts, so that `+` can mean addition of reals, complex numbers or vectors. `BRA` defines functions written with brackets, such as a scalar product $\langle a, b \rangle$.

Each sort has its own representation in C. Real numbers are `double` values; the standard library defines complex numbers, real vectors and matrices, whose values are C arrays. Operations on whole vectors and matrices are executed by C functions, so they run at compiled speed.

2.2 Rules and algebras

Rules are collected in named rule sets, called *algebras*. A rule has a left side, a right side, an optional condition and an optional block of instructions that is executed before the replacement:

```
VAR A : real
ALG simp
SPEC    simp : A*1 = A              # SPEC starts a new rule set
ADDSPEC simp : A+0 = A              # ADDSPEC adds a rule
CONST u : real
REWR (u*1)+0 : simp                 # prints u
```

`VAR` declares meta variables that match any term of their sort. `NUM` declares meta variables that only match numbers, which makes rules like "evaluate if the argument is a number" easy to write. Conditions test whether a term occurs in another one (`IN`, `NIN`), compare terms in a total order (`LB`, used for sorting sums), or evaluate any expression with a boolean result.

Algebras are combined into lists. A list in parentheses is applied repeatedly until no rule changes the term any more. The standard rule list of the library is defined like this:

```
SPEC All : Std,Doit,(Linear,Compl,diff,GrRi),AGroup,integr,
```

```
        (Subst,GrRi),Std,Zus,unaerweg
```

With the library loaded, `REWR D(sin cos x,x) :  All` returns `- cos (cos (x ))* sin (x )`.

2.3 Transferring rules with LIFT

A distinctive feature of `rewrite` is the `LIFT` statement. It declares that all rules valid for one set of functions and constants also hold for another set. The standard library writes the laws of a group once and transfers them step by step:

```
LIFT (Ag+Ag,Ag-Ag,-Ag,0g) TO (Aag+Aag,Aag-Aag,-Aag,0ag)
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar+Ar,Ar-Ar,-Ar,0r)
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar*Ar,Ar/Ar,mi(Ar),1r)
```

The last line reuses the rules of an abelian group for multiplication, division, the multiplicative inverse `mi` and the neutral element 1 of a ring. In the same way, linearity is written once for a generic linear function and lifted to differentiation, integration, Taylor expansion and summation. Each symbol can be lifted from one other symbol, so the lifted operators form a tree, and a rule applies to a term whose operator reaches the operator of the rule along this tree. This corresponds closely to the algebraic hierarchies of modern proof assistants [15], but here it acts directly on the rewrite rules.

2.4 Exact numbers as subsorts

Exact arithmetic was added in 2026 without changing the rules of the library, by two sorts and two new statements. The sorts `integer` (arbitrary size) and `rational` (fractions) are implemented in C with the GMP library [7] and declared in an include file:

```
SORT integer
SORT rational
ATT integer  show_int,cmp_int,cpy_int,del_int
ATT rational show_rat,cmp_rat,cpy_rat,del_rat
SUBSORT integer,rational ATT int2rat     # every integer is a fraction
SUBSORT rational,real ATT rat2real       # every fraction is a real number
OTHER + : rational,rational >> rational ATT rat_plus
OTHER / : integer,integer >> rational  ATT int_div
...
LIFT (A+A,A-A,A*A,A/A,-A,mi(A),A^A) TO (Aq+Aq,Aq-Aq,Aq*Aq,Aq/Aq,-Aq,mi(Aq),Aq^Az)
INTNUM natural ATT read_int              # 12 is a natural number, 0.5 real
```

`SUBSORT` declares that every value of the first sort is also a value of the second, with a C function that converts the value. Sorts remain static: the sort of a term follows from its parts when it is built, and a value never changes its sort at run time. A function declared for a wider sort accepts arguments of its subsorts; if several variants of an operator fit, the one with the fewest conversions is chosen, and the values are converted only when the function is calculated. Thus `1/3+1/6` uses the rational `+` and prints `(1/2)`, `2^100` is exact, and `1/3+0.5` uses the real `+` and prints `0.833333`. A variant that fits exactly is always preferred, so that conversions are only a fallback, as the most specific method in Julia: the library declares `real` a subsort of `complex`, and the hand-written variant for `real + complex` still wins over converting the real number. Two variants that need equally many conversions (a natural number fits both an `integer` and a `nonneg` variant) are reported, and a more specific variant resolves the tie. The `LIFT` line lets `rational` inherit all rules of `real`, and a second one does the same for `integer`; `x*1+0*x+(1/2)*x` simplifies to `(3/2) * x` with the unchanged rules for real numbers.

Symbolic numbers. Above `real` and `complex` the library declares the sort `number` of symbolic scalar expressions, like `Number` in Julia. Its operator variants have no C function; a term is of sort `number` only when one of its parts is, for example the imaginary unit `im`, which has no numerical value and of which the symbolic helpers only know $im^2 = -1$. A meta variable of sort `number` matches any term, and a function in a pattern whose variant is declared on such top sorts also matches its variants on narrower sorts, as a method for `Number` in Julia applies to `Float64`. The right-hand side and the conditions of a rule are evaluated with the variants for the actual sorts, so a rule written for `number` still calculates `m > 0` for a real `m`. The derivative and the translated Rubi rules are written this way and apply to real integrands and to integrands with `im` alike, while a real integrand keeps its real operators.

Sorts with a known sign. A sort may have several wider sorts, and the conversion follows the shortest path in the resulting graph. The library declares

$$\mathit{natural} < \mathit{integer} < \mathit{rational} < \mathit{real}, \qquad \mathit{natural} < \mathit{nonneg} < \mathit{real}, \qquad \mathit{positive} < \mathit{nonneg},$$

where `nonneg` and `positive` are real numbers known to be non-negative or positive. Since the minus sign is an operator, every number literal is `natural`, and `2-5` is the `integer` -3. Operator variants such as `nonneg*nonneg » nonneg` and `sqrt : nonneg » nonneg` propagate the sign through terms. Several simplification rules are valid only for non-negative or positive arguments: $\sqrt{t^2} = t$, $(t^a)^b = t^{ab}$, $\ln(pq) = \ln p + \ln q$. They are now written with meta variables of these sorts. A meta variable whose sort is a subsort of another only matches terms of that sort or below it, whereas a variable of a sort without a wider sort still matches any term as in the original language. Thus `sqrt(t^2)` simplifies to `t` for `CONST t : nonneg`, but `sqrt(x^2)` stays as it is for a real `x`.

One change in the interpreter made this work. When a rule written for `real` rewrites a term with exact numbers, the operators of the new term are chosen again for the sorts of their actual arguments, from the leaves upwards and after each numerical step. The original `LIFT` mechanism replaced all operators of one sort by those of another; that was correct for homogeneous structures like groups and rings, but with subsorts the arguments of one term have different sorts. Re-choosing the variant repairs this and makes rules for `real` produce exact results whenever their inputs are exact. The declarations are a classic instance of order-sorted algebra [6]; the novelty here is that the subsort relation, the conversions and the inheritance of rules are all defined in the library.

Roots of exact numbers follow Mathematica. `4^(1/2)` is the integer 2, but `2^(1/2)` has no exact value: the C function for a rational power returns nothing, and the power stays a term inside another term. Such terms are kept in a normal form: perfect powers are taken out and exponents reduced ($\sqrt{8} = 2\sqrt{2}$, $\sqrt{4/3} = 2 \cdot 3^{-1/2}$, $(\sqrt{2})^2 = 2$), and in a simplified product a coefficient is merged with a root of the same base ($2/\sqrt{2} = \sqrt{2}$). This needed a distinction between two kinds of numerical evaluation that were the same before: subterms of a term are calculated exactly or not at all, while the value of a closed expression, for output or for a numerical test in a rule condition, falls back to real arithmetic ($\sqrt{2}$ prints 1.41421). A real number in an operation makes it numeric, as Mathematica's numeric contagion: $\sqrt{2} \cdot 0.5$ is 0.707107, whereas a real 1.0 next to an exact number counts as the integer 1, since the rules of the old library were written with real numbers.

2.5 Interleaving rewriting and numerics

After every single rewrite step, the interpreter evaluates all subterms whose arguments are numbers. Constants that have a value are replaced by their value when a term is copied. Together with the procedural part of the language (`FOR`, `WHILE`, `IF`, blocks with local constants, and user functions), this allows numerical algorithms to be written as rewrite rules. The library defines Simpson integration as a rule whose `DO` block computes the value:

```
ALG rechne
SPEC rechne : I(N,M,A,_x)=_myerg DO { _myerg=0;_zw1=0;_zw2=0  # Simpson
        FOR n=N+_step/2 TO M-_step STEP _step DO { _x=n;_zw1=_zw1+A
        _x=n+_step/2
        _zw2=_zw2+A }
        _x=M-_step/2
        _zw1=_zw1+A
        _x=N;_myerg=A+4*_zw1+2*_zw2
        _x=M;_myerg=(_myerg+A)*_step/2/3
        _x=dummi }
```

A second rule set first substitutes the integration variable by the internal variable `_x`. With `_step=0.01`, `REWR I(0,pi,sin(x),x) : numerik,rechne` returns 2. The same mechanism gives Newton's method and the search for minima and maxima. The symbolic part can prepare the numerical part: `newton` expects the derivative as an argument, which is computed symbolically by `D`.

The original application of the language was physics. The library contains physical constants, units and formulas, and an example program computes Bragg angles, structure factors and X-ray absorption of lithium niobate crystals from tabulated atomic data.

3 Case study: rule-based integration

Differentiation is easy to express with rewrite rules, because every rule takes the term apart and the product and chain rules always apply. Integration is harder. Complete decision procedures exist for elementary functions [12, 2], but they are not pattern based. Rubi [11, 13] showed that an extensive system of carefully ordered rules with conditions can integrate a very large class of functions. Its 199 rule files contain 7422 rules in Mathematica syntax, published under the MIT license. We first translated a small closed subset by hand and then wrote a translator for all of them, to test how far the mechanisms of `rewrite` carry.

3.1 Pattern features

Rubi's patterns rely on features of the Mathematica pattern matcher that the original `rewrite` did not have. A typical rule is

```
Int[(a_. + b_.*x_)^m_, x_Symbol] := (a + b*x)^(m + 1)/(b*(m + 1)) /;
    FreeQ[{a, b, m}, x] && NeQ[m, -1]
```

where `a_.` matches a missing summand with default 0 and `b_.` a missing factor with default 1. The pattern also matches $x\,b + a$, because Mathematica matches sums and products in any order, and the condition combines several tests. We added three corresponding features to the language: `OPT a,b` declares meta variables that may be missing, `DEFAULT + = 0`, `DEFAULT * = 1` and `DEFAULT RIGHT ^ = 1` give the values of missing operands, `COMM +,*` declares commutative operators, and a rule can have a list of conditions, all of which must hold. `NIN(x,a,b,m)` tests that x occurs in none of a, b, m, which is Rubi's `FreeQ`.

A first implementation expanded every rule into all its variants when it was defined, so that the matcher stayed unchanged. The number of variants grows exponentially with the number of optional parameters and commutative products, however: the median Rubi rule would have about 1900 variants. Rules defined while one of these features is active are therefore kept in an ordered *rule list*, as in Mathematica, and matched by a separate backtracking matcher. Bindings of meta variables are recorded on a trail, so a failed alternative undoes exactly its own bindings. Swapped operands and missing optional operands are tried in turn, and the conditions are checked as part of the match, so a failing condition makes the matcher try the next alternative

and then the next rule. A precomputed set of possible head symbols per rule skips most rules of a large list without running the matcher. With these features the rule above translates almost one to one:

```
ADDSPEC RubiRules : IF NIN(Xv,O_a,O_b), (neq(P_m,(-1)))
        Int(((O_a)+((O_b)*(Xv)))^(P_m),Xv) = ...
```

Here `P_m` is a required and `O_a`, `O_b` are optional meta variables. One rule covers $(a+bx)^m$, $(bx+a)^m$, $(a+x)^m$, $(bx)^m$ and x^m, with numeric or symbolic coefficients.

3.2 Automatic translation

The translator (900 lines of Python) parses Rubi's rule files with a small Pratt parser into Mathematica's full form and writes one `rewrite` file per Rubi file, in Rubi's loading order and with the original of every rule as a comment. Pattern variables become meta variables, negative numbers in patterns become `NUM` variables with a condition, local definitions (`With`, and `Module` with values only) are substituted, helper predicates that Rubi defines inside its rule files are inlined, and a predicate applied to a list, like `LinearQ[{u,v},x]`, becomes a conjunction. As in Mathematica, e^u is a power whose base is a symbolic constant e of sort `positive` without a numerical value, so that the rules for F^u apply to it and $\ln e = 1$ stays exact. Some rules have a function variable, `F_[c_.*(a_.+b_.*x_)]` with a condition such as `TrigQ[F]` or `MemberQ[{Sinh,Cosh},F]`; they are expanded into one rule per admissible function.

Helper and special functions. Rubi's conditions and right-hand sides use about a hundred helper functions, such as `ExpandIntegrand`, `PolyQ`, `BinomialQ`, `Subst` or `ExpandTrigReduce`. They cannot be ordinary `rewrite` functions, which are only evaluated when all arguments are numbers. We added symbolic C functions instead: they receive the argument terms and return a new term, or nothing if they do not apply. About 90 of them are implemented, from predicates on the form of a term to polynomial division, the product-to-sum reduction of trigonometric powers, and Rubi's measures of simplicity based on the size of a term. Their results are exact when their inputs are.

Such functions break with the principle that the interpreter knows nothing about the meaning of an operator: a C function that expands a product must know which symbol is the product. To keep this dependence explicit, the helpers are declared with `ATT SYMBOLIC` and refer to the library only through *roles* such as `"+"`, `"cos"` or `"pi"`. By default a role is played by the symbol of the same name; a library with other names maps them (`SYMBOLIC "cos" = "kos"`). A head is recognized by its function family, so that the variants of an operator for different sorts count as one. What the roles cannot express are the algebraic properties the C code relies on, such as the commutativity of `+`; these are documented as the contract of each role. Where a helper only tests the form of a term, the matcher can do the work instead: `matchq(u, pattern, cond)`, Rubi's `MatchQ`, runs the backtracking matcher of the rule lists on a pattern with its own variables and checks the condition with their bindings. It is declared with `HOLD`, so that the pattern and the condition reach it unevaluated, and it made 28 more rules translatable. Rubi's own form tests, such as `LinearMatchQ` or `TrinomialMatchQ`, are defined in its utility file with `MatchQ`; the translator now inlines these definitions instead of using C functions for them.

Knowledge that a helper needs beyond a name, such as the derivative of the sine, can be kept in rules as well: a function declared with `ATT RULES alglist` is calculated by rewriting its call with the rule list, also inside a condition and from C. The derivative used by `DerivativeDivides` is such a function, defined with the rules for `D` of the library; Rubi's `FixInertTrigFunction` is another, translated from Rubi's definitions (44 of 60 are used: in Rubi they act on inert functions only, which the translation does not distinguish, and 12 of them would take integrands away from the rules that solve them). A rewriting started in the middle of a condition must set the

bindings of the rule being tried aside and restore them afterwards; with that, the interpreter supports rule-defined functions at any depth. `ExpandIntegrand` splits rational functions into partial fractions, as Mathematica's `Apart`: with GMP rationals, the denominator is split into square-free parts, rational roots and the factors given in the input, and the extended Euclidean algorithm gives the numerators. Rubi's last general rule, $\int u\,dx = \int \texttt{ExpandIntegrand}[u]\,dx$ if the result is a sum, then integrates every rational function term by term. A condition like `SumQ[ExpandIntegrand[u,x]]` needs its inner helper evaluated first; `rewrite` calculated conditions from the outside in, so the rule never applied until this was changed. Rubi's substitutions rest on `FunctionOfQ` and `SubstFor`, which decide whether the integrand is a function of, say, $\sin w$ and write it in a new variable, also through identities such as $\cos^2 w = 1 - \sin^2 w$; with these two C functions, 34 substitution rules for trigonometric and hyperbolic functions became translatable. `DerivativeDivides[y,u,x]` returns u/y' if it is free of x, and `False` otherwise; it is a C function that stays unevaluated when it does not apply, and `FalseQ` tests for exactly that, so that 20 rules of the form $\int u\,f(y)\,dx = q\int f(t)\,dt$ with $t = y$ could be translated. The substitution $x \to F^{gx}$ for functions of exponentials is a C function that takes the place of Rubi's rule, which uses pattern tests the translator cannot express; so are the substitution $t = (a+bx)^{1/n}$ for fractional powers of a linear function and Euler's substitutions for $\sqrt{a+bx+cx^2}$, whose Rubi rules pass a list of results. A small difference had a large effect: Rubi's `GtQ[u,v]` is false unless both are numbers, so that `Not[GtQ[a^2,0]]` holds for a symbol a; translated as $u > v$, the comparison could not be calculated, and rules such as $\int dx/\sqrt{a^2-x^2} = \arctan(x/\sqrt{a^2-x^2})$ never applied. With a helper of Rubi's meaning, 30 more problems were solved. Many results of Rubi need special functions; 27 of them, from the error function and the Fresnel and exponential integrals to the incomplete Gamma function, elliptic integrals in Carlson's form [3] and the hypergeometric function ${}_2F_1$, were added as C functions with Mathematica's conventions, each with a derivative rule. Their values agree with mpmath [9] to within 10^{-11}.

The host system's simplification. Rubi relies on Mathematica's automatic simplification, which `rewrite` does not have. Most of the work after the first translation went into reproducing the parts Rubi depends on. A normal-form rule set writes $x - y$ as $x + (-1)\,y$, x/y as $x\,y^{-1}$, $\sqrt{x}$ as $x^{1/2}$ and $(uv)^k$ as $u^k v^k$ for integer k (in the translated patterns as well), but, unlike our first version and like Mathematica, it does not multiply out products with sums: Rubi's rules for $(c+d\tan x)/(a+b\tan x)$ expect the product. Polynomials in the integrand are brought into the form $c_0 + c_1 x + c_2 x^2 + \ldots$ with collected coefficients and their numeric content in front, and every rule result is simplified trivially (u^0, u^1, $1\cdot u$, $0+u$). In Mathematica, some rules that make no progress are harmless because exact arithmetic or other rules prevent a loop; in `rewrite` they looped. The rule list therefore skips a rule whose result equals the term, contains a NaN or a division by zero, or closes a cycle such as expanding and factoring again (the result was replaced before, and the chain of the last replacements, also of subterms, leads back to the term), and a rule that keeps rewriting the same term. Only the chain counts: a term that was replaced in another branch, like $\int b\,dx$, may come up again. The matcher also had to learn that an optional variable already bound to its default value may be missing a second time, as in $(c + d\tan(e+fx))/(a+b\tan(e+fx))$ with $e = 0$, and that products are flat: Mathematica's pattern $u\,(a+b\arctan x)$ matches $x^2\arctan x/(1+x^2)$ with $u = x^2/(1+x^2)$, so for a commutative operator the matcher also tries every split of up to six nested operands into two groups. Finally, Rubi's substitutions $\int u\,dx$ with $x \to \sqrt{a+bx}$ rely on Mathematica treating the new variable as non-negative. `Subst[Int[u,x],x,v]` therefore integrates in a fresh constant of sort `nonneg` when v is a square root or otherwise non-negative, so that $\sqrt{x^2} = x$ holds inside; with this, $\int \sin\sqrt{1+x}/\sqrt{1+x}\,dx = -2\cos\sqrt{1+x}$. Mathematica also writes $(b\tan u)^3$ as $b^3\tan^3 u$, so a pattern $(b\,\tan u)^n$ with integer n always sees $b = 1$ there, and some rules silently drop b. The translator adds the condition $b = 1$ (or $b = 0$ in a sum) to every rule whose optional variable appears neither in a condition nor in the result.

Two parts of Rubi lie outside its rule files. Most trigonometric rules exist only for sine, secant and tangent; Rubi's utility function `UnifyInertTrigFunction` first rewrites cosine forms, such as $a + b\cos(e + fx)$ as $a + b\sin(e + \pi/2 + fx)$. Its 69 definitions are translated by the same program into rules that run before the trigonometric ones, and the output rules turn $\sin(\pi/2 + u)$ back into $\cos u$. Rubi also relies on Mathematica writing $1/\sin x$ as $\csc x$, and on its own `FixInertTrigFunction`, which rewrites products of two trigonometric functions, e.g. $\sec x \sin^2 x$ as $\cos^{-1} x \sin^2 x$; a C function with Rubi's tables does both before the rules are tried. Without it, even $\int dx/\sin x$ remained unsolved. Similarly, functions of trigonometric functions of different multiples of one angle, such as $\sin x/\cos 2x$ or $e^{\sin x}\sin 2x$, are written with $\sin x$ and $\cos x$ first, as Mathematica's `TrigExpand` does; polynomials in sines and cosines are left to Rubi's product-to-sum reduction, which the expansion would undo. For hyperbolic functions, Rubi uses $\sinh u = -i\sin(iu)$, which needs complex numbers. Instead, a last rule writes hyperbolic functions for which no other rule applies with exponentials, $\sinh u = (e^u - e^{-u})/2$, so that $\int x \sinh x\, dx$ and $\int \cosh^2 x\, dx$ are found.

The complete integration is the algebra list

```
SPEC RubiIntegrate : (RubiNorm,IntRubi,RubiRules),RubiCollect,IntOut,RubiOut
```

where `RubiCollect` combines like summands of the result, `IntOut` restores the usual output form and `RubiOut` writes the result in a normal form for output in one pass of a C function. The library's general simplification, which did this before, sorts and simplifies one rule application at a time; for a polynomial result of 231 terms it needed 24 000 steps, each followed by a new traversal of the term, and took most of the time of such integrals. As Rubi, `rewrite` can show the solution steps: after `STEPS RubiRules "file"`, every rule of the list that is applied is printed with the integral before and after it, labelled with its Rubi section, number and Mathematica original from a file that the translator writes along with the rules.

3.3 Results

The translator converts 7183 of Rubi's 7492 rules (96%; rules that Rubi writes in two variants, with and without `ShowStep`, count twice), which become 7627 `rewrite` rules after the expansion of function variables. Rules that Rubi writes as small programs (`Module` with several statements, `Sum`, lists) are replaced by C functions: the substitutions of section 9.3 and the partial fractions of $x^m/(a+bx^n)$ with the coefficients $\cos(k\pi/n)$. The largest group of the remaining rules needs complex numbers (245 rules); others use the rest of Rubi's substitution machinery (`FunctionOfExponentialQ`), general pattern tests (`MatchQ`), function variables without a finite set of functions, or the generalized hypergeometric function. Loading all translated rules takes about half a second, and a typical integral a fraction of a second. Table 1 shows examples.

To measure the translation, a test harness runs problems of Rubi's own test suite [14] through `RubiIntegrate`. It does not compare the result with Rubi's antiderivative, which may differ by a constant or in form, but checks it numerically: the derivative of the result, computed by a five-point difference quotient, must equal the integrand at several points where the integrand is real, with fixed values for symbolic parameters. The four collections of independent problems in the test suite (from the textbooks of Apostol, Hearn, Stewart and Timofeev, 1540 problems, of which 1539 use only functions that `rewrite` has) gave the results in Table 2.

These collections come from textbooks and are the problems on which the translation was developed. A sample of every 20th problem of the other sections of the test suite (about 4600 problems, of 72 000) shows where the translation is less complete: algebraic functions 88%, trigonometric functions 71%, exponentials 67%, hyperbolic functions 51%, inverse hyperbolic functions 47%, logarithms 34% and inverse trigonometric functions 30% are solved. Part of the difference is due to the problems themselves: 3–23% of those in a section need complex numbers in Rubi's result, and some cannot be integrated by Rubi either. The rest points to missing or

Integrand	Result of `REWR Int(f,x) : RubiIntegrate`
`1/(1+x^3)`	`(1/3)*ln(1+x) + (1/3)*3^(1/2)*arctan(-1/3^(1/2)+2*x/3^(1/2)) - (1/6)*ln(1+x^2-x)`
`x/(1+x^4)`	`(1/2)*arctan(x^2)`
`sqrt(1-x^2)`	`(1/2)*arcsin(x) + (1/2)*(1-x^2)^(1/2)*x`
`x*sqrt(1+x)`	`(2/5)*(1+x)^(5/2) - (2/3)*(1+x)^(3/2)`
`sin(x)^2*cos(x)^3`	`(1/3)*sin(x)^3 - (1/5)*sin(x)^5`
`1/(1+tan(x))^2`	`(1/2)*ln(cos(x)+sin(x)) - (1/2)/(1+tan(x))`
`x*arcsec(x)`	`(1/2)*x^2*arcsec(x) - (1/2)*(1-1/x^2)^(1/2)*x`
`exp(-x^2)`	`0.886227*erf(x)`
`sin(x)/x`	`sinint(x)`
`1/sqrt(1-x^4)`	`ellipticf(arcsin(x),-1)`
`(1+x^3)^(1/3)`	`hyp2f1((-1/3),(1/3),(4/3),-x^3)*x`

Table 1: Indefinite integrals computed with the automatically translated Rubi rules (output slightly condensed). Rational coefficients and roots such as $1/\sqrt{3}$ are exact; $\sqrt{\pi}/2$ is a real number, since π is.

Result	Problems	Share
solved and verified	1392	90.4%
no antiderivative found	123	8.0%
wrong result	3	0.2%
crash or time-out	0	0.0%
not verifiable	21	1.4%

Table 2: Results on the 1539 tested problems of four independent integration problem collections in Rubi's test suite.

inexact helpers; a single wrong predicate (`PolyQ` with a non-symbol as variable) cost a fifth of the solved logarithm problems.

The development of the translation went through several such measurements; each round of fixes, such as the normal form without multiplying out, the matching of optional variables bound to their default, the special functions and the missing helpers, raised the share of verified results. During a first measurement, 41% of the problems were solved, 55% before the cosine unification, the hyperbolic fallback and the symbolic e, 62% after them, 79% after partial fractions, the inside-out evaluation of nested helpers and the flat matching of products, 84% with Rubi's substitutions, 85.5% with the normal form of trigonometric products, and 88% with Rubi's meaning of `GtQ` and the substitutions for roots, and 90% with the expansion of multiple angles. Two of the results counted as wrong contain special functions evaluated outside their real domain (${}_2F_1$ at an argument $e^{7x}/3 > 1$, the incomplete Gamma function of a negative argument), where the check compares different complex branches; the third writes $\sqrt{\sin x \cos^3 x}$ as $\cos^{3/2} x \, \sin^{1/2} x$, which is wrong where both factors are negative. A run takes about five minutes.

The translation also served as a test of the library itself. It revealed two incorrect simplification rules in the standard algebra, which turned $\ln(x+y)$ into $\ln x \cdot \ln y$ and a^{bc} into a^{bc}, two incorrect derivatives (of arcsin and arccos), and three rules that are only valid for positive bases: $(ab)^c = a^c b^c$, $(a^b)^c = a^{bc}$, which turned $\sqrt{x^2}$ into x, and $\ln(ab) = \ln a + \ln b$. They now apply only for integer exponents or a positive numerical factor, and for terms of the sorts `nonneg` and `positive`. The numerical check of the harness found these errors, because Rubi's results were differentiated at negative arguments.

4 Implementation and revival

The interpreter consists of about 8800 lines of C, of which about 3900 were added in 2026 for the rule lists, subsorts, exact numbers, special and helper functions, and a grammar of about 490 lines for the scanner and parser generators flex and bison. Terms are binary trees of nodes taken from memory pools; symbols carry their sort, value and the links used by `LIFT`. The rules of an algebra are merged into a single tree, in which rules with a common left-hand-side structure share nodes and alternatives branch off where they differ; rule lists are kept in order instead. Rules are matched against every subterm, innermost first, and a single `REWR` stops after 10^5 steps, so that a looping rule set cannot hang the interpreter.

The program was written for 32-bit Unix and the Amiga and no longer worked with current tools. The port to 64-bit Linux and Windows (MSYS2) required only small changes to the C code itself. The main problems were changed semantics in the scanner generator, the release of static memory that old C libraries tolerated, and errors in the handling of declarations inside loops and included files. A regression test suite with 28 test programs now covers the core language, the library, the pattern features, exact numbers, helper and special functions, the integration rules and the original example programs; the integration tests check every result by differentiating it again. Memory leaks were removed with an instrumented build that counts allocations per source line, so that repeated calculations run in constant memory. All parser structures are now accessed through named macros instead of chains like `np->r->r->l`; for most source files the compiled code stayed byte-identical, which confirmed that this cleanup did not change the behaviour. `rewrite` runs interactively with a prompt, runs program files given as arguments, or reads a program from a pipe for scripts and the tests.

Performance is adequate for interactive use. Loading the standard library and computing a derivative takes well below 0.1 s on a current laptop; loading the 7329 integration rules about half a second. A scalar loop written in the language itself needs about 0.25 μs per iteration (measured for 10^7 iterations of `s = s + k*k`). We estimate this to be about two orders of magnitude slower than an equivalent compiled loop. Operations on whole vectors and matrices run in C.

5 Discussion

Strengths. The main strength of `rewrite` is that all mathematics is visible and changeable. Correcting the derivative of arcsin meant editing one line of a text file, not the interpreter. `LIFT` allows algebraic structure to be expressed once and reused; with subsorts, it also turned out to be the right mechanism for a numeric tower, since exact fractions inherited all rules of the real numbers without any change to them. `NUM` variables together with conditions give precise control over when a rule fires. The interleaving of rewriting and numerical evaluation makes it natural to derive formulas symbolically and evaluate them numerically in the same program, for example derivatives for Newton's method.

Limitations. Commutative matching only swaps the two operands of one binary operator. Associativity is handled by normalization, so the integrand must be brought into a fixed form before the rules are tried. The imaginary unit is a symbol of the sort `number`, but the general simplification of the library is still written for real numbers, and logarithms of negative arguments evaluate to NaN although their derivatives are correct. The sign of a term is only known through the sorts of its operator variants, so for example e^x is not known to be positive. Nothing checks that a rule set terminates or is confluent [1], and the order of rules matters; the safeguards against loops in rule lists are heuristics. Numeric conditions such as $m \neq -1$ do not hold for symbolic m, which is more conservative than Rubi. Error messages are minimal.

Related work. Rule-based simplification is at the core of Mathematica [17] and of Rubi [11]. General rewriting logic, with order-sorted signatures, is provided by Maude [5, 6]. More recently, equality saturation with e-graphs, as in egg [16] and Metatheory.jl [4], avoids the dependence on rule order by representing many equivalent terms at once. Proof assistants organize mathematical knowledge in algebraic hierarchies and simplification rule sets [15]. `rewrite` anticipated several of these ideas in a small and self-contained system.

Outlook. Natural next steps are complex numbers on top of the number tower, which would also open the 245 Rubi rules with the imaginary unit but require complex versions of the helper and special functions; Rubi's substitution machinery; an associative-commutative matcher for sums and products of any length; and code generation that turns symbolically derived expressions into compiled numerical code.

6 Conclusion

`rewrite` shows that a small interpreter can host a complete computer algebra library when the mathematics is expressed as rewrite rules and rule sets can be combined and lifted between algebraic structures and number sorts. The automatic translation of 96% of Rubi demonstrates that established rule-based algorithms can be imported, and it clarifies which matching features and which implicit simplifications of the host system such an import requires.

Availability

The source code, the standard library, the integration rules, the translator, the test harness and the test suite are available at `https://github.com/RainerHeintzmann/rewrite` under the MIT license [8]. The translated integration rules are derived from Rubi [13] and keep its MIT license notice.

Acknowledgements

The scanner and parser framework of the original program is based on code by Axel T. Schreiner. The 2026 port, the test suite, the memory leak analysis, the exact numbers, the special functions and the translation of the integration rules were carried out with the assistance of an AI coding assistant (Claude, Anthropic).

References

[1] Franz Baader and Tobias Nipkow. *Term Rewriting and All That.* Cambridge University Press, 1998.

[2] Manuel Bronstein. *Symbolic Integration I: Transcendental Functions.* Springer, 2 edition, 2005.

[3] B. C. Carlson. Numerical computation of real or complex elliptic integrals. *Numerical Algorithms*, 10:13–26, 1995.

[4] Alessandro Cheli. Metatheory.jl: Fast and elegant algebraic computation in Julia with extensible equality saturation. *Journal of Open Source Software*, 6(59):3078, 2021.

[5] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn Talcott. *All About Maude – A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science.* Springer, 2007.

[6] Joseph A. Goguen and José Meseguer. Order-sorted algebra I: equational deduction for multiple inheritance, overloading, exceptions and partial operations. *Theoretical Computer Science*, 105(2):217–273, 1992.

[7] Torbjörn Granlund and the GMP development team. GNU MP: The GNU multiple precision arithmetic library. `https://gmplib.org`, 2023. Version 6.3.

[8] Rainer Heintzmann. rewrite – a computer language that first has to learn mathematics from include files. `https://github.com/RainerHeintzmann/rewrite`.

[9] Fredrik Johansson et al. mpmath: a Python library for arbitrary-precision floating-point arithmetic. `https://mpmath.org`, 2023. Version 1.3.0.

[10] Aaron Meurer et al. SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3:e103, 2017.

[11] Albert Rich, Patrick Scheibe, and Nasser M. Abbasi. Rule-based integration: An extensive system of symbolic integration rules. *Journal of Open Source Software*, 3(32):1073, 2018.

[12] Robert H. Risch. The problem of integration in finite terms. *Transactions of the American Mathematical Society*, 139:167–189, 1969.

[13] Rule-Based-Integration Organization. Rubi – rule-based integrator, version 4.17.3. `https://github.com/RuleBasedIntegration/Rubi`. MIT license.

[14] Rule-Based Integration Organization. Rubi integration test suite in Mathematica syntax. `https://github.com/RuleBasedIntegration/MathematicaSyntaxTestSuite`, 2018. MIT license.

[15] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020.

[16] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages*, 5(POPL), 2021.

[17] Wolfram Research, Inc. Mathematica. Champaign, IL, `https://www.wolfram.com/mathematica`.